

Laboratory assignment

Component 3

Authors: Patricia Moga, Ciobanu Sergiu-Tudor

Group: 5

May 28, 2026

1 Conceptual Modeling: The PAGES System

The development of this Multi-Agent System (MAS) follows the PAGES framework: Perception, Action, Goal, Environment, and State of the Environment. This approach ensures a transition from abstract requirements to concrete agent behaviors.

1.1 PAGES for each Agent

Each agent within the system acts as a distinct entity with specific roles and capabilities:

Agent	Perception (P)	Action (A)	Goal (G)	Env / State
Coordinator	Unassigned patients in MapModel.	Initiating auctions (CFP), Selecting winners.	Assign all patients efficiently.	Global Registry of missions and assignments.
Ambulance	CFPs, Traffic alerts, current coordinates.	A* Bidding, movement, re-calculating detour.	Reach the patient efficiently.	Dynamic Road Network with moving entities
Traffic	Ambulance trajectories (activePaths).	Generating blockages, broadcasting alerts.	Simulate dynamic constraints.	Current paths of active emergency vehicles
Tracing	traveled paths, original bids.	Data recording, efficiency reporting.	Evaluate agent performance.	Historical data.

Table 1: PAGES Framework per Agent Type

2 Environment Properties

The environment is a discrete 25x25 grid representing city infrastructure. It is composed of static tiles (Roads, Obstacles) and dynamic elements (Ambulance positions, blockages).

- **Type:** The environment is **Stochastic** due to random traffic blockages and **Dynamic** as it changes during agent reasoning.
- **Accessibility:** While partially observable for individual agents, it is **Accessible** via the MapModel shared state.

Agent	Accessibility	Interaction Type
Ambulance	Local / Shared	Active: Navigates and modifies state.
Traffic Agent	Shared (Limited)	Reactive: Injects blockages based on paths.
Coordinator	Global	Deliberative: Views entire patient list.
Tracing Agent	Global (History)	Passive: Analyzes finished trajectories.

Table 2: Agent-Environment Accessibility

3 Design of the Application

The application architecture is structured to separate environment state, visualization, and agent orchestration into distinct layers.

- **MapModel:** This class initializes the static infrastructure of the city, specifically the road network and obstacles. By hard-coding these elements as static, the system ensures reliable pathfinding and prevents the randomization from creating unreachable start or end points. Once the static grid is established, the class dynamically assigns random starting coordinates for the ambulances and the patients from the available road tiles.
- **TerrainView:** This class relies on the grid from `MapModel` to construct the city’s visual representation. It renders squares to depict the state of each coordinate, linked by a road network that supports horizontal, vertical, and diagonal movement. The `drawTerrain` function updates the dynamic elements—such as moving vehicles and trajectories—every one second, as given by the GUI’s timer.
- **GUI:** The GUI serves as the primary dispatcher interface. It includes a real-time visualization of the map, a status monitor displaying current ambulance coordinates, and a mission log container that displays auction results and performance traces. To initiate the simulation, the GUI calls the `ConfigApp` class to spawn the agent container.
- **ConfigApp:** This class manages the system lifecycle. It first calls the static initialization method in `MapModel` to instantiate the grid, define the number of agents, and place patients. Only after the environment is ready does it create and start the JADE agents.

4 Communication and Interaction Model

The agent interaction model follows a strict hierarchy of perception, deliberation, and feedback.

- **Coordination Logic:** The `AmbulanceCoordinatorAgent` perceives unassigned patients in the `MapModel` and initiates an auction via a Call for Proposal (CFP). An `AmbulanceAgent` will ignore a CFP if it is already moving to another patient. Each free ambulance uses its internal `GpsSystem` to calculate an A* path from its current coordinates to the patient, returning the total step count. The Coordinator collects all bids and identifies the winner by selecting the proposal with the minimum value, representing the shortest possible path at that moment.
- **Reactive Navigation:** While an `AmbulanceAgent` follows its planned path, the `TrafficAgent` monitors the trajectory to inject dynamic blockages. When a blockage is detected, the `TrafficAgent` sends a `Traffic-Alert` (INFORM), forcing the ambulance to recalculate its route from its current coordinate to the patient.

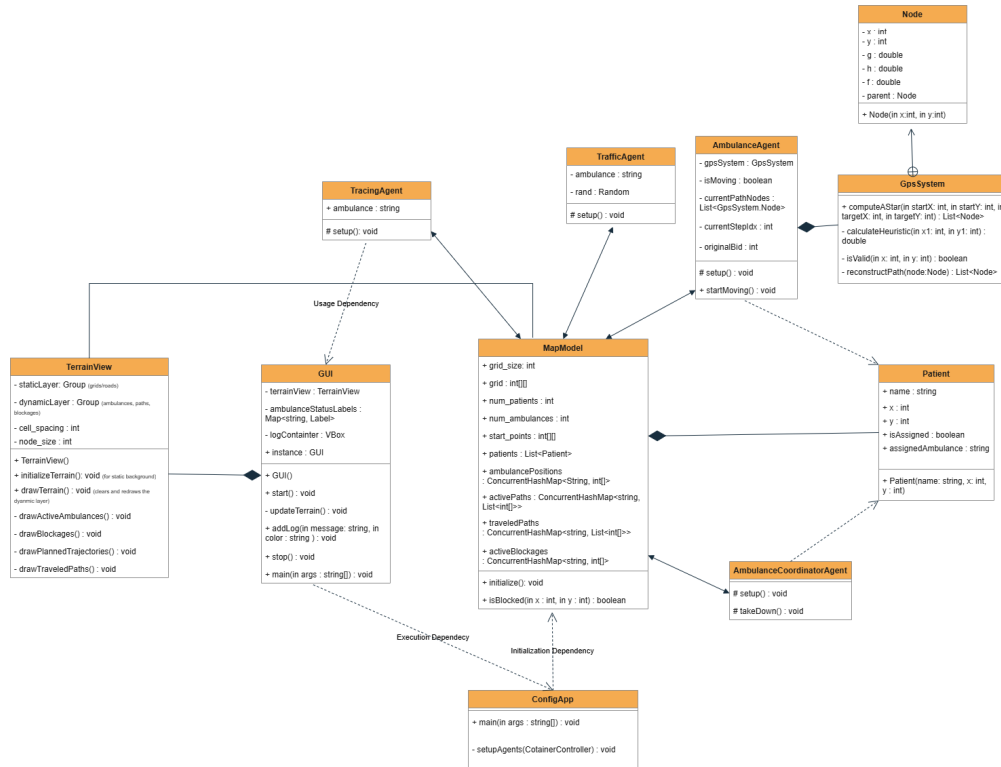


Figure 1: Class Diagram

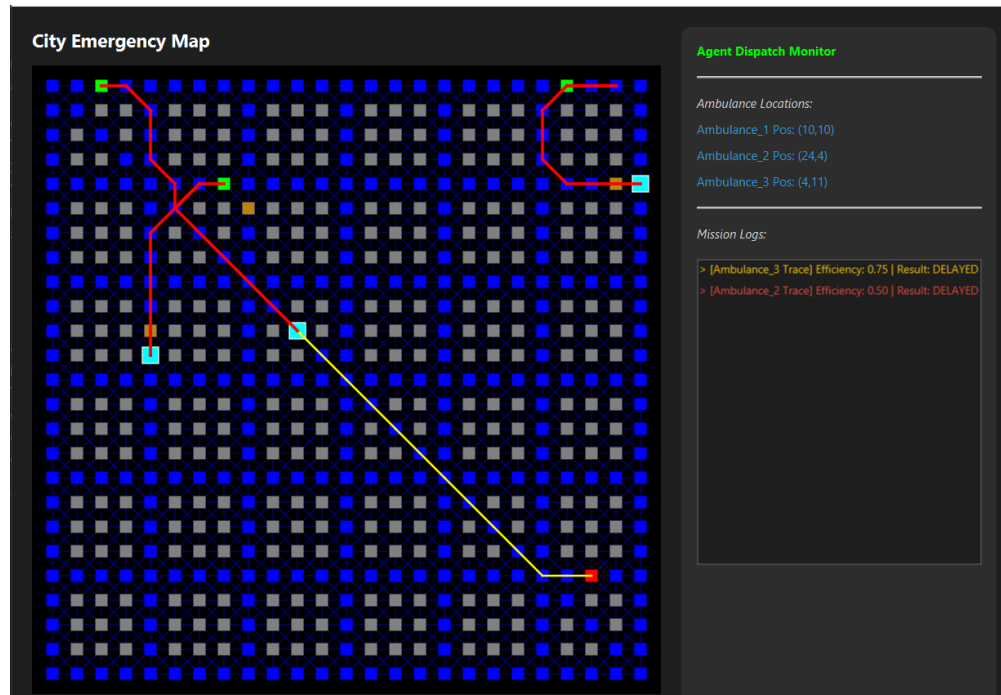


Figure 2: Dispatcher Simulation Visualization

- **Performance Feedback:** Upon reaching the target, the ambulance notifies its dedicated `TracingAgent`. The `TracingAgent` then calculates the mission efficiency by comparing the original bid against the actual steps taken, pushing the final report to the GUI's log container.